

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет программной инженерии и компьютерных технологий

**Лабораторная работа №4
По предмету: “Вычислительная математика”
Вариант № 11**

Выполнил: Федоров Д. А.

Р3206

Проверил: Зинченко А. А.

г. Санкт-Петербург 2026 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ 3

1. 3

2. 3

3. 3

4. 4

Исходный код программы:..... 4

Результаты работы программы:..... 5

Заключение:..... 7

ЗАДАНИЕ:

Цель лабораторной работы: найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов.

Лабораторная работа состоит из двух частей: вычислительной и программной.

№ варианта задания лабораторной работы определяется как номер в списке группы согласно ИСУ.

Порядок выполнения работы

Вычислительная реализация задачи

Вычислительная часть лабораторной работы должна быть представлена только в отчете.

Задание:

1. Сформировать таблицу табулирования заданной функции на указанном интервале (см. табл. 1)
2. Построить линейное и квадратичное приближения по 11 точкам заданного интервала;
3. Найти среднеквадратические отклонения для каждой аппроксимирующей функции. Ответы дать с тремя знаками после запятой;
4. Выбрать наилучшее приближение;
5. Построить графики заданной функции, а также полученные линейное и квадратичное приближения;
6. Привести в отчете **подробные вычисления**.

Программная реализация задачи

Для исследования использовать:

- линейную функцию,
- полиномиальную функцию 2-й степени,
- полиномиальную функцию 3-й степени,
- экспоненциальную функцию,
- логарифмическую функцию,
- степенную функцию.

Методика проведения исследования:

1. Вычислить меру отклонения: $S = \sum_{i=1}^n [\varphi(x_i) - y_i]^2$ для всех исследуемых функций;
2. Уточнить значения коэффициентов эмпирических функций, минимизируя функцию S;
3. Сформировать массивы предполагаемых эмпирических зависимостей $(\varphi(x_i), \varepsilon_i)$;
4. Определить среднеквадратичное отклонение для каждой аппроксимирующей функции. Выбрать наименьшее значение и, следовательно, наилучшее приближение;
5. Построить графики полученных эмпирических функций.

Задание:

1. Предусмотреть ввод исходных данных из файла/консоли (таблица $y = f(x)$ должна содержать от 8 до 12 точек);
2. Реализовать метод наименьших квадратов, исследуя все указанные функции;
3. Предусмотреть вывод результатов в файл/консоль: коэффициенты аппроксимирующих функций, среднеквадратичное отклонение, массивы значений $x_i, y_i, \varphi(x_i), \varepsilon_i$;
4. Для линейной зависимости вычислить коэффициент корреляции Пирсона;
5. Вычислить коэффициент детерминации, программа должна выводить соответствующее сообщение в зависимости от полученного значения R^2 ;
6. Программа должна отображать наилучшую аппроксимирующую функцию;
7. Организовать вывод графиков функций, графики должны полностью отображать весь исследуемый интервал (с запасом);
8. Программа должна быть протестирована при различных наборах данных, в том числе и некорректных;

Вычислительная часть:

1. Линейная аппроксимация:

$$y = \frac{5x}{x^4 + 11} \quad x \in [0, 4] \quad h = 0,2$$

i	x_i	y_i	$\varphi(x_i)$	$(\varphi - y_i)^2$
1	0	0	0,295	0,087
2	0,4	0,181	0,283	0,010
3	0,8	0,351	0,272	0,006
4	1,2	0,459	0,260	0,040
5	1,6	0,456	0,249	0,043
6	2	0,37	0,237	0,018
7	2,4	0,272	0,225	0,002
8	2,8	0,193	0,214	0,000
9	3,2	0,138	0,202	0,004
10	3,6	0,101	0,191	0,008
11	4	0,075	0,179	0,011

$$\varphi(x, a, b) = a \cdot x + b$$

$$SX = \sum x_i = 0,0 + 0,4 + 0,8 + 1,2 + 1,6 + 2,0 + 2,4 + 2,8 + 3,2 + 3,6 + 4,0 = 22$$

$$SXX = \sum x_i^2 = 0 + 0,16 + 0,64 + 1,44 + 2,56 + 4 + 5,76 + 7,84 + 10,24 + 12,96 + 16 = 61,6$$

$$SY = \sum y_i = 0 + 0,181 + 0,351 + 0,459 + 0,456 + 0,37 + 0,272 + 0,193 + 0,138 + 0,101 + 0,075 = 2,596$$

$$SXY = \sum x_i \cdot y_i = 0 + 0,072 + 0,281 + 0,551 + 0,73 + 0,74 + 0,653 + 0,54 + 0,442 + 0,364 + 0,3 = 4,673$$

$$\begin{cases} a \cdot SXX + b \cdot SX = SXY \\ a \cdot SX + b \cdot n = SY \end{cases}$$

$$\begin{cases} a \cdot 61 + b \cdot 22 = 4,673 \\ a \cdot 22 + b \cdot 11 = 2,596 \end{cases}$$

$$\Delta = SXX \cdot n - SX \cdot SX = 61,6 \cdot 11 - 22 \cdot 22 = 193,6$$

$$\Delta_1 = SXY \cdot n - SX \cdot SY = 4,673 \cdot 11 - 22 \cdot 2,596 = -5,709$$

$$\Delta_2 = SXX \cdot SY - SX \cdot SXY = 61,600 \cdot 2,596 - 22,000 \cdot 4,673 = 57,108$$

$$a = \frac{\Delta_1}{\Delta} = -\frac{5,709}{193,6} = -0,029$$

$$b = \frac{\Delta_2}{\Delta} = \frac{57,108}{193,6} = 0,295$$

$$\varphi(x) = -0,029 \cdot x + 0,295$$

$$\sigma = \sqrt{\frac{\sum (\varphi(x_i) - y_i)^2}{n}} = 0,144$$

2. Квадратичная аппроксимация:

i	x _i	y _i	φ(x _i)	(φ - y _i) ²
1	0	0	0,081	0,007
2	0,4	0,181	0,195	0
3	0,8	0,351	0,281	0,005
4	1,2	0,459	0,34	0,014
5	1,6	0,456	0,371	0,007
6	2	0,37	0,375	0
7	2,4	0,272	0,352	0,006
8	2,8	0,193	0,3	0,011

9	3,2	0,138	0,221	0,007
10	3,6	0,101	0,114	0
11	4	0,075	-0,019	0,009

$$\varphi(x, a_0, a_1, a_2) = a_0 + a_1 \cdot x + a_2 \cdot x^2$$

$$SX = \Sigma x_i = 22$$

$$SXX = \Sigma x_i^2 = 61,6$$

$$SY = \Sigma y_i = 2,596$$

$$SXY = \Sigma x_i y_i = 4,673$$

$$SX^3 = \Sigma x_i^3 = 0 + 0,064 + 0,512 + 1,728 + 4,096 + 8 + 13,824 + 21,952 + 32,768 + 46,656 + 64 = 193,6$$

$$SX^4 = \Sigma x_i^4 = 0 + 0,026 + 0,41 + 2,074 + 6,554 + 16,000 + 33,178 + 61,466 + 104,858 + 167,962 + 256 = 648,528$$

$$SX^2Y = \Sigma x_i^2 \cdot y_i = 0 + 0,029 + 0,225 + 0,661 + 1,167 + 1,48 + 1,567 + 1,513 + 1,413 + 1,309 + 1,2 = 10,564$$

$$\begin{cases} n \cdot a_0 + SX \cdot a_1 + SX^2 \cdot a_2 = SY \\ SX \cdot a_0 + SX^2 \cdot a_1 + SX^3 \cdot a_2 = SXY \\ SX^2 \cdot a_0 + SX^3 \cdot a_1 + SX^4 \cdot a_2 = SX^2Y \end{cases}$$

$$\begin{cases} 11 \cdot a_0 + 22 \cdot a_1 + 61,6 \cdot a_2 = 2,596 \\ 22 \cdot a_0 + 61,6 \cdot a_1 + 193,6 \cdot a_2 = 4,673 \\ 61,600 \cdot a_0 + 193,6 \cdot a_1 + 648,528 \cdot a_2 = 10,564 \end{cases}$$

$$\Delta = 4\,253,005; \Delta_0 = 346,145; \Delta_1 = 1\,357,441; \Delta_2 = -367,378$$

$$a_0 = \frac{\Delta_0}{\Delta} = 0,081$$

$$a_1 = \frac{\Delta_1}{\Delta} = 0,319$$

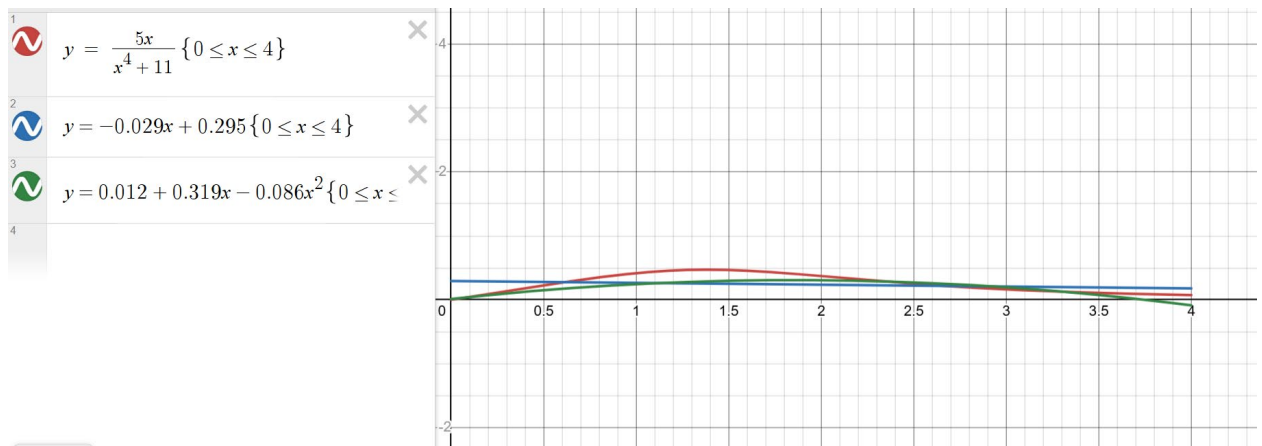
$$a_2 = \frac{\Delta_2}{\Delta} = -0,086$$

$$a_0 = 0,081; a_1 = 0,319; a_2 = -0,086$$

$$\varphi(x) = 0,081 + 0,319 \cdot x - 0,086 \cdot x^2$$

$$\sigma = \sqrt{\frac{\sum (\varphi(x_i) - y_i)^2}{n}} = 0,078$$

$0,078 < 0,144 \Rightarrow$ среднеквадратичное отклонение меньше у линейного, значит квадратичное приближение лучше



Программная часть:

Листинг программы:

```
import math

import matplotlib.pyplot as plt

import numpy as np

def solve_linear_system(A, B):

    n = len(A)

    augmented = np.hstack([A, B.reshape(-1, 1)]).astype(float)

    for i in range(n):

        max_row = i + np.argmax(np.abs(augmented[i:, i]))

        augmented[[i, max_row]] = augmented[[max_row, i]]

    if abs(augmented[i, i]) < 1e-10:
```

```
return None
```

```
augmented[i] = augmented[i] / augmented[i, i]
```

```
for j in range(n):
```

```
    if j != i:
```

```
        augmented[j] = augmented[j] - augmented[j, i] * augmented[i]
```

```
return augmented[:, -1]
```

```
def read_from_file(filename="data.txt"):
```

```
    try:
```

```
        x, y = [], []
```

```
        with open(filename, 'r', encoding='utf-8') as f:
```

```
            for line in f:
```

```
                line = line.strip()
```

```
                if not line:
```

```
                    continue
```

```
                line = line.replace(',', ' ')
```

```
                parts = line.split()
```

```
                if len(parts) != 2:
```

```
                    continue
```

```
                x.append(float(parts[0]))
```

```
                y.append(float(parts[1]))
```

```
    if len(x) == 0:
```

```
        print("Ошибка: файл не содержит корректных данных")
```

```
    return None, None
```

```

x = np.array(x)

y = np.array(y)

if len(x) < 8 or len(x) > 12:

    print(f"Предупреждение: {len(x)} точек (рекомендовано 8-12)")

    return x, y

except Exception as e:

    print(f"Ошибка чтения: {e}")

    return None, None

def read_from_console():

    print("Введите x y (каждая пара на новой строке, пустая строка - конец)")

    x, y = [], []

    while True:

        line = input().strip()

        if not line:

            if len(x) >= 8:

                break

            else:

                print(f"Нужно минимум 8 точек, введено {len(x)}")

                continue

        line = line.replace(',', ' ')

        parts = line.split()

        if len(parts) != 2:

            print("Нужно два числа")

            continue

        try:

```

```
        x.append(float(parts[0]))
        y.append(float(parts[1]))
    except:
        print("Ошибка ввода")
    return np.array(x), np.array(y)
```

```
def pearson_correlation(x, y):
    n = len(x)
    x_mean = np.sum(x) / n
    y_mean = np.sum(y) / n

    numerator = np.sum((x - x_mean) * (y - y_mean))
    denominator = np.sqrt(np.sum((x - x_mean)**2) * np.sum((y - y_mean)**2))

    return numerator / denominator if denominator != 0 else 0
```

```
def r_squared(y_true, y_pred):
    ss_res = np.sum((y_true - y_pred)**2)
    ss_tot = np.sum((y_true - np.mean(y_true))**2)
    return 1 - (ss_res / ss_tot) if ss_tot != 0 else 1
```

```
def plot_data_and_functions(x, y, results, best_result):
    x_min, x_max = np.min(x), np.max(x)
    margin = (x_max - x_min) * 0.1
    x_plot = np.linspace(x_min - margin, x_max + margin, 200)

    plt.figure(figsize=(12, 7))
    plt.scatter(x, y, color='black', s=50, zorder=5, label='Исходные данные')
```

```

colors = ['red', 'blue', 'green', 'orange', 'purple', 'brown']

for i, res in enumerate(results):

    if res['func']:

        y_plot = res['func'](x_plot)

        plt.plot(x_plot, y_plot, color=colors[i % len(colors)],

                 linewidth=2, label=f"{res['name']} ( $\delta$ = {res['std_dev']:.4f})")

plt.xlabel('x')

plt.ylabel('y')

plt.title('Аппроксимация методом наименьших квадратов')

plt.legend(loc='best', fontsize=9)

plt.grid(True, alpha=0.3)

info = f"Лучшая: {best_result['name']}\n $\delta$  = {best_result['std_dev']:.6f}\n $R^2$  = {best_result['r2']:.4f}"

plt.text(0.02, 0.98, info, transform=plt.gca().transAxes, fontsize=10,

        verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

plt.tight_layout()

plt.show()

def linear_approx(x, y):

    n = len(x)

    SX = np.sum(x)

    SXX = np.sum(x**2)

    SY = np.sum(y)

    SXY = np.sum(x * y)

    delta = SXX * n - SX * SX

```

```
delta1 = SXY * n - SX * SY
```

```
delta2 = SXX * SY - SX * SXY
```

```
a = delta1 / delta
```

```
b = delta2 / delta
```

```
def func(xi): return a * xi + b
```

```
y_pred = func(x)
```

```
std_dev = np.sqrt(np.sum((y_pred - y)**2) / n)
```

```
r2 = r_squared(y, y_pred)
```

```
return {
```

```
    'name': 'Линейная (ax + b)',
```

```
    'params': (a, b),
```

```
    'param_names': ('a', 'b'),
```

```
    'func': func,
```

```
    'std_dev': std_dev,
```

```
    'r2': r2,
```

```
    'y_pred': y_pred
```

```
}
```

```
def quadratic_approx(x, y):
```

```
    n = len(x)
```

```
    SX = np.sum(x)
```

```
    SXX = np.sum(x**2)
```

```
    SXXX = np.sum(x**3)
```

```
    SXXXX = np.sum(x**4)
```

```
    SY = np.sum(y)
```

```
SXY = np.sum(x * y)
```

```
SXXY = np.sum((x**2) * y)
```

```
A = np.array([[n, SX, SXX],
```

```
              [SX, SXX, SXXX],
```

```
              [SXX, SXXX, SXXXX]])
```

```
B = np.array([SY, SXY, SXXY])
```

```
try:
```

```
    a0, a1, a2 = solve_linear_system(A, B)
```

```
    if a0 is None:
```

```
        return None
```

```
except:
```

```
    return None
```

```
def func(xi): return a0 + a1 * xi + a2 * xi**2
```

```
y_pred = func(x)
```

```
std_dev = np.sqrt(np.sum((y_pred - y)**2) / n)
```

```
r2 = r_squared(y, y_pred)
```

```
return {
```

```
    'name': 'Полином 2-й степени ( $a_0 + a_1 \cdot x + a_2 \cdot x^2$ )',
```

```
    'params': (a0, a1, a2),
```

```
    'param_names': ('a0', 'a1', 'a2'),
```

```
    'func': func,
```

```
    'std_dev': std_dev,
```

```
    'r2': r2,
```

```
    'y_pred': y_pred
```

```
}
```

```
def cubic_approx(x, y):
```

```
    n = len(x)
```

```
    SX = np.sum(x)
```

```
    SXX = np.sum(x**2)
```

```
    SXXX = np.sum(x**3)
```

```
    SXXXX = np.sum(x**4)
```

```
    SXXXXX = np.sum(x**5)
```

```
    SXXXXXX = np.sum(x**6)
```

```
    SY = np.sum(y)
```

```
    SXY = np.sum(x * y)
```

```
    SXXY = np.sum((x**2) * y)
```

```
    SXXXY = np.sum((x**3) * y)
```

```
    A = np.array([[n, SX, SXX, SXXX],
```

```
                  [SX, SXX, SXXX, SXXXX],
```

```
                  [SXX, SXXX, SXXXX, SXXXXX],
```

```
                  [SXXX, SXXXX, SXXXXX, SXXXXXX]])
```

```
    B = np.array([SY, SXY, SXXY, SXXXY])
```

```
    try:
```

```
        a0, a1, a2, a3 = solve_linear_system(A, B)
```

```
        if a0 is None:
```

```
            return None
```

```
    except:
```

```
        return None
```

```
def func(xi): return a0 + a1 * xi + a2 * xi**2 + a3 * xi**3
```

```

y_pred = func(x)

std_dev = np.sqrt(np.sum((y_pred - y)**2) / n)

r2 = r_squared(y, y_pred)

return {
    'name': 'Полином 3-й степени ( $a_0 + a_1 \cdot x + a_2 \cdot x^2 + a_3 \cdot x^3$ )',
    'params': (a0, a1, a2, a3),
    'param_names': ('a0', 'a1', 'a2', 'a3'),
    'func': func,
    'std_dev': std_dev,
    'r2': r2,
    'y_pred': y_pred
}

```

```

def exponential_approx(x, y):

```

```

    if np.any(y <= 0):
        return None

```

```

    ln_y = np.log(y)

```

```

    n = len(x)

```

```

    SX = np.sum(x)

```

```

    SXX = np.sum(x**2)

```

```

    SY = np.sum(ln_y)

```

```

    SXY = np.sum(x * ln_y)

```

```

    delta = SXX * n - SX * SX

```

```

    delta1 = SXY * n - SX * SY

```

```

    delta2 = SXX * SY - SX * SXY

```

```
B = delta1 / delta
```

```
A = delta2 / delta
```

```
a = math.exp(A)
```

```
b = B
```

```
def func(xi): return a * np.exp(b * xi)
```

```
y_pred = np.array([func(xi) for xi in x])
```

```
std_dev = np.sqrt(np.sum((y_pred - y)**2) / n)
```

```
r2 = r_squared(y, y_pred)
```

```
return {
```

```
    'name': 'Экспоненциальная ( $a \cdot e^{(b \cdot x)}$ )',
```

```
    'params': (a, b),
```

```
    'param_names': ('a', 'b'),
```

```
    'func': func,
```

```
    'std_dev': std_dev,
```

```
    'r2': r2,
```

```
    'y_pred': y_pred
```

```
}
```

Вывод:

В данной лабораторной работе я изучил аппроксимацию функций методом наименьших квадратов. Были рассчитаны линейное и квадратичное приближение функции, а также написана программа для аппроксимирования функции несколькими методами.